

Definición de recursos **RESTFUL**

Programación Orientada a Objetos – ISAM 2025

REST

- REST: *“Transferencia de Estado Representacional”*
- Es un estilo de arquitectura de software, utilizado generalmente en API's y aplicaciones de cliente – servidor que define recursos y su separación en distintos tipos de operaciones.
- En Laravel, utilizamos recursos RESTful al definir las rutas y los controladores bajo una serie de operaciones separadas para cada recurso.

¿Qué es un recurso?

- Elemento definido que compone parte de la lógica de la aplicación, generalmente identificado por un sustantivo.
- Ejemplos:
 - Usuarios
 - Grupos
 - Transacciones
 - Mensajes

Estructura de recursos REST

Verb	URI	Action	Route Name
GET	/photos	index	photos.index
GET	/photos/create	create	photos.create
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
GET	/photos/{photo}/edit	edit	photos.edit
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Resumen de cada acción

- **index**: muestra un listado de todos los elementos del recurso
- **create**: muestra la vista necesaria para ingresar los datos de un nuevo elemento
- **store**: guarda y crea el nuevo elemento
- **show**: muestra la vista de detalle de un elemento específico
- **edit**: muestra la vista necesaria para actualizar un elemento
- **update**: guarda y actualiza el elemento seleccionado
- **destroy**: elimina un elemento específico

Rutas de un recurso REST

- Dos opciones:
 - 1) Definir las rutas a mano, teniendo cuidado de incluir cada uno de los métodos y nombres apropiados para cada ruta.
 - 2) Utilizar el método `resource()` que define automáticamente las 7 rutas necesarias.

```
Route::resource('cursos', CursosController::class);
```

```
php artisan route:list
```

Crear controlador REST

- Al crear un controlador podemos definir que se deben agregar los 7 métodos de recursos:

```
php artisan make:controller TransaccionesController  
--resource
```

- El nuevo controlador creado tendrá los 7 métodos REST definidos.

Parchar métodos PUT, PATCH y DELETE

- Los métodos PUT, PATCH y DELETE no existen nativamente en HTML, por ello deberemos parcharlos en los formularios que los usan.
- Para ello definimos el formulario como POST y luego usamos internamente la directiva @method de Blade.

```
<form action="..." method="POST">  
    @csrf  
    @method("PATCH")  
    <!-- Resto del formulario -->  
</form>
```


Agenda

- Con el recurso de transacciones, haremos lo siguiente:
 - Definir las 7 rutas para cada acción
 - Transformar TransaccionesController en un controlador REST
 - Crear las vistas show y edit
 - Modificar la vista index y agregar:
 - Botón para actualizar una transacción
 - Botón para eliminar una transacción (confirmar acción)